

Implémentation, ServeurMultiThreade (v1)

Fabrice.Kordon@lip6.fr



Architecture

2



Deux systèmes de communication (classiques)

-  Clients / serveur
 - ▶ File de messages (partagée)
-  Serveurs / servants et servants / client
 - ▶ Appel de méthode (dédié)

La classe Requete

3

```
import java.util.Random;
import java.util.concurrent.locks.*;

class Requete {
    UnClient emetteur;
    int valeur;

    Requete(UnClient c, int val) {
        valeur = val;
        emetteur = c;
    }

    public String toString () {
        return "(" + emetteur + ", " + valeur + ")" ;
    }
}
```

Contenu d'une requête

Valeur + référence sur le client

La classe FileRequetes

4

```
class FileRequetes {
    private Requete tableau[];
    private int premiere, derniere; // pointeurs sur requêtes
    private int occupe; // nombre de requêtes contenues

    private final ReentrantLock l = new ReentrantLock();
    private final Condition ecrire = l.newCondition();
    private final Condition lire = l.newCondition();

    FileRequetes(int taille) {
        tableau = new Requete[taille];
        premiere = 0;
        derniere = 0;
        occupe = 0;
    }

    private void trace (String m) {
        System.out.println ("FileRequetes : " + m);
    }

    public String toString() {
        return "[" + occupe + ", " + premiere + ", " + derniere + "]";
    }
}
```


La classe FileRequetes

4

```
public void insererRequete (Requete r) {  
    l.lock();  
    try {  
        while (occupe == tableau.length) {  
            this.trace("file pleine pour " + r);  
            try {  
                ecrire.await();  
            }  
            catch (InterruptedException e) {  
                System.err.println("Ne doit pas arriver (1)");  
            }  
        }  
        tableau[premiere] = r;  
        occupe++;  
        premiere++;  
        if (premiere == tableau.length) {  
            premiere = 0;  
        }  
        lire.signalAll();  
    }  
    finally {  
        l.unlock();  
    }  
}
```

La classe FileRequetes

4

```
public Requete extraireRequete () {
    Requete vret;
    l.lock();
    try {
        while (occupe == 0) {
            this.trace("file vide");
            try {
                lire.await();
            }
            catch (InterruptedException e) {
                System.err.println("Ne doit pas arriver (2)");
            }
        }
        vret = tableau[derniere];
        occupe--;
        derniere++;
        if (derniere == tableau.length) {
            derniere = 0;
        }
        ecrire.signalAll();
    }
    finally {
        l.unlock();
    }
    return vret;
}
```


La classe UnClient

5

```
class UnClient implements Runnable {
    private int monId;
    private FileRequetes f;
    private Random generateur = new Random();
    private final Object reveil = new Object(); // attente réponse
    private static int cpt = 1;
    private static final Object mutex = new Object();

    UnClient (FileRequetes fr) {
        f = fr;
        synchronized (mutex) {
            monId = cpt++;
        }
    }

    private void trace (String m) {
        System.out.println ("UnClient " + monId + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public void requeteTraitee () {
        synchronized (reveil) {
            reveil.notifyAll();
        }
    }

    public String toString () {
        return "client " + monId;
    }
}
```


La classe UnClient

5

```
public void run() {  
    this.trace("initialisé");  
    Requete r = new Requete(this, generateur.nextInt(1000));  
    this.trace("j'envoie la requête " + r);  
    f.insererRequete (r);  
    this.trace("j'attends le serveur");  
    try {  
        synchronized (veille) {  
            veille.wait();  
        }  
    }  
    catch (InterruptedException e) {  
        System.out.println ("UnClient "+monId+", cela doit jamais arriver, " + e);  
    }  
    this.trace("ma requête a été traitée");  
}
```

}

La classe Servant

6

```
class Servant implements Runnable {
    private Requete maRequete;

    Servant (Requete r) {
        maRequete = r;
    }

    private void trace (String m) {
        System.out.println ("Servant " + maRequete + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public void run() {
        this.trace("initialisé");
        try {
            Thread.sleep(maRequete.valeur);
        }
        catch (InterruptedException e) {
            System.out.println ("Servant, ne doit jamais arriver, " + e);
        }
        this.trace("je préviens le client");
        maRequete.emetteur.requeteTraitee();
    }
}
```


La classe LeServeur

7

```
class LeServeur implements Runnable {
    private FileRequetes f;

    LeServeur (FileRequetes fr) {
        f = fr;
    }

    private void trace (String m) {
        System.out.println ("Serveur : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public void run() {
        Requete r;
        this.trace("initialisé");
        while (true) {
            r = f.extraireRequete(); // potentiellement bloquant
            this.trace("je traite " + r);
            new Thread (new Servant(r)).start();
        }
    }
}
```


La «classe principale»

8

```
public class ServeurMultiThreade1 {  
  
    static int tailleFile;  
    static int nbClients;  
  
    public static void main(String []args) {  
        try {  
            tailleFile = Integer.parseInt(args[0]);  
            nbClients = Integer.parseInt(args[1]);  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.err.println ("Arguments requis, taille de la file, nombre de clients.");  
            return;  
        }  
        // Lancer le système  
        FileRequetes laFile = new FileRequetes(tailleFile);  
        for (int i = 0; i < nbClients ; i++) {  
            new Thread(new UnClient(laFile)).start();  
        }  
        new Thread(new LeServeur(laFile)).start();  
    }  
}
```


Des exécutions

Terminaison?

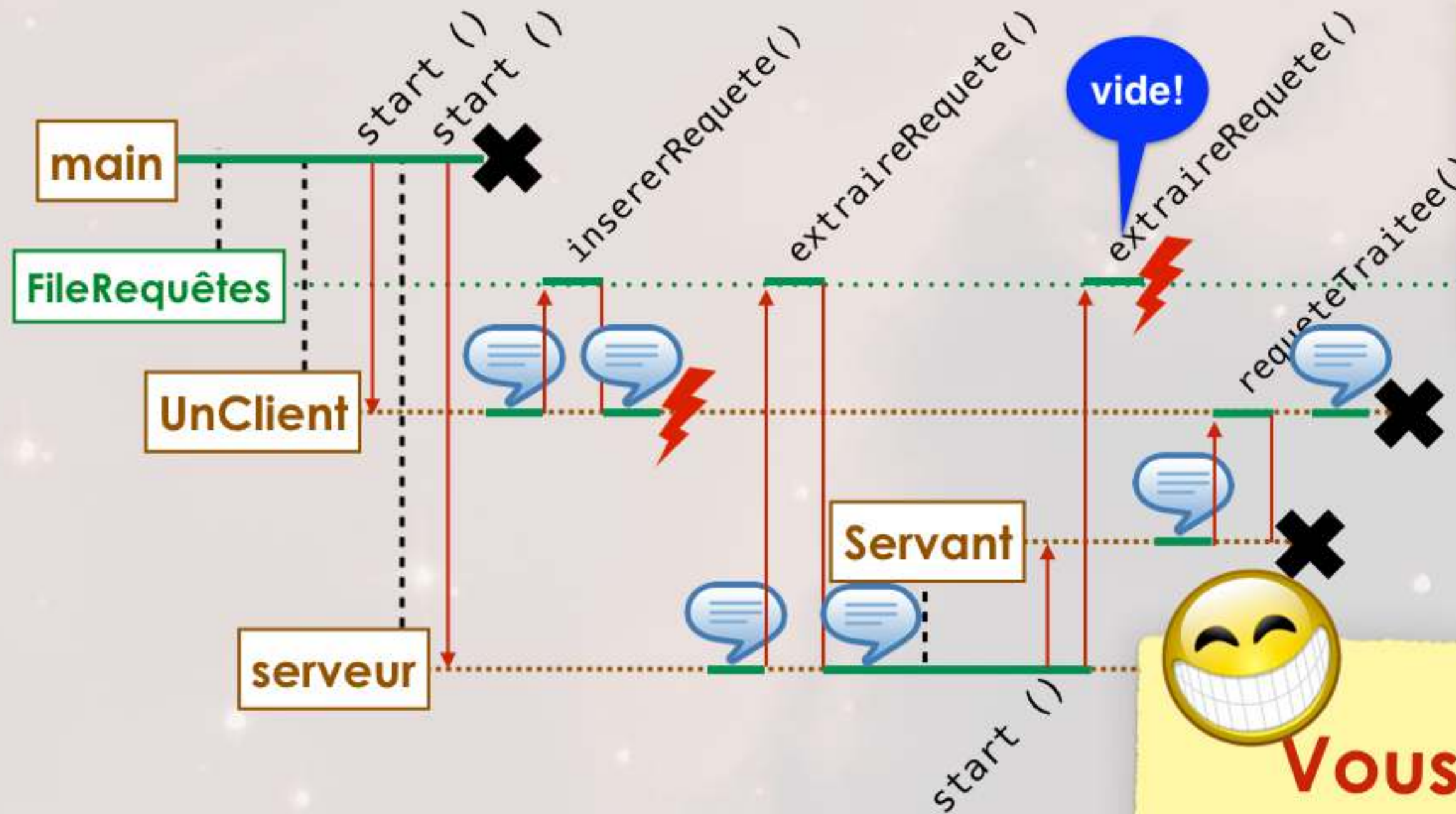
«à la main» pour le serveur

```
$ java ServeurMultiThreade1 2 3
UnClient 1 : initialisé
UnClient 3 : initialisé
UnClient 2 : initialisé
UnClient 1 : j'envoie la requête (client 1, 377)
UnClient 3 : j'envoie la requête (client 3, 659)
UnClient 2 : j'envoie la requête (client 2, 332)
Serveur : initialisé
UnClient 2 : j'attends le serveur
Serveur : je traite (client 1, 377)
UnClient 1 : j'attends le serveur
UnClient 3 : j'attends le serveur
Serveur : je traite (client 2, 332)
Servant (client 1, 377) : initialisé
Serveur : je traite (client 3, 659)
Servant (client 2, 332) : initialisé
FileRequetes : file vide
Servant (client 3, 659) : initialisé
Servant (client 2, 332) : je préviens le client
UnClient 2 : ma requête a été traitée
Servant (client 1, 377) : je préviens le client
UnClient 1 : ma requête a été traitée
Servant (client 3, 659) : je préviens le client
UnClient 3 : ma requête a été traitée
^C
```

```
$ java ServeurMultiThreade1 1 3
UnClient 1 : initialisé
UnClient 3 : initialisé
UnClient 2 : initialisé
UnClient 3 : j'envoie la requête (client 3, 239)
UnClient 2 : j'envoie la requête (client 2, 534)
UnClient 1 : j'envoie la requête (client 1, 468)
Serveur : initialisé
UnClient 3 : j'attends le serveur
FileRequetes : file pleine pour (client 2, 534)
FileRequetes : file pleine pour (client 1, 468)
Serveur : je traite (client 3, 239)
UnClient 2 : j'attends le serveur
FileRequetes : file pleine pour (client 1, 468)
Serveur : je traite (client 2, 534)
UnClient 1 : j'attends le serveur
Serveur : je traite (client 1, 468)
Servant (client 3, 239) : initialisé
FileRequetes : file vide
Servant (client 1, 468) : initialisé
Servant (client 2, 534) : initialisé
Servant (client 3, 239) : je préviens le client
UnClient 3 : ma requête a été traitée
Servant (client 1, 468) : je préviens le client
UnClient 1 : ma requête a été traitée
Servant (client 2, 534) : je préviens le client
UnClient 2 : ma requête a été traitée
^C
```


Un petit chronogramme pour la route?

10



Ici...

pas de blocage de la file
(1 client = 1 requête)



Vous avez compris?

En exercice, avec 2 clients

```
$ java ServeurMultiThreade1 1 1
UnClient 1 : initialisé
UnClient 1 : j'envoie la requête (client 1, 689)
UnClient 1 : j'attends le serveur
Serveur : initialisé
Serveur : je traite (client 1, 689)
FileRequetes : file vide
Servant (client 1, 689) : initialisé
Servant (client 1, 689) : je prévient le client
UnClient 1 : ma requête a été traitée
```

^C